

UNIVERSIDAD AMERICANA
FACULTAD DE INGENIERÍA Y ARQUITECTURA



Servicios Web

VIII Semestre

Tarea Semana 7

Evidencias Práctica Guiada

Estudiante:

Kendrick Kaleh Barrios Padilla

Docente: MSc. José Alejandro Durán García

Managua, 29 de septiembre de 2026

Configuración de la conexión

```
Producto.java application.properties Proveedor.java Guardar categorias.yml .gitignore
1 spring.application.name=gestion-productos
2
3 spring.datasource.url=jdbc:postgresql://localhost:5432/gestion_productos
4 spring.datasource.username=postgres
5 spring.datasource.password=12345678
6
7 spring.jpa.hibernate.ddl-auto=validate
8 spring.jpa.show-sql=true
9 spring.jpa.properties.hibernate.format_sql=true
10
11 spring.flyway.enabled=true
12
```

Estructura de paquetes



Entidades desarrolladas

1. Categoria

```
@Entity @kendrickbarrios
@Table(name = "categoria")
public class Categoria {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String nombreCategoria; 2 usages

    private Boolean activa; 2 usages
```

2. Producto

```
@Entity @kendrickbarrios
@Table(name = "producto")
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String codigo; 2 usages

    private String nombre; 2 usages

    @ManyToOne 2 usages
    @JoinColumn(name = "categoria_id")
    private Categoria categoria;

    @ManyToOne 2 usages
    @JoinColumn(name = "proveedor_id")
    private Proveedor proveedor;

    private String descripcion; 2 usages

    @Column(name = "precio_venta") 2 usages
    private BigDecimal precioVenta;

    @Column(name = "existencia") 2 usages
    private int existencia;
```

3. Proveedor

```
@Entity @kendrickbarrios
@Table(name = "proveedor")
public class Proveedor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

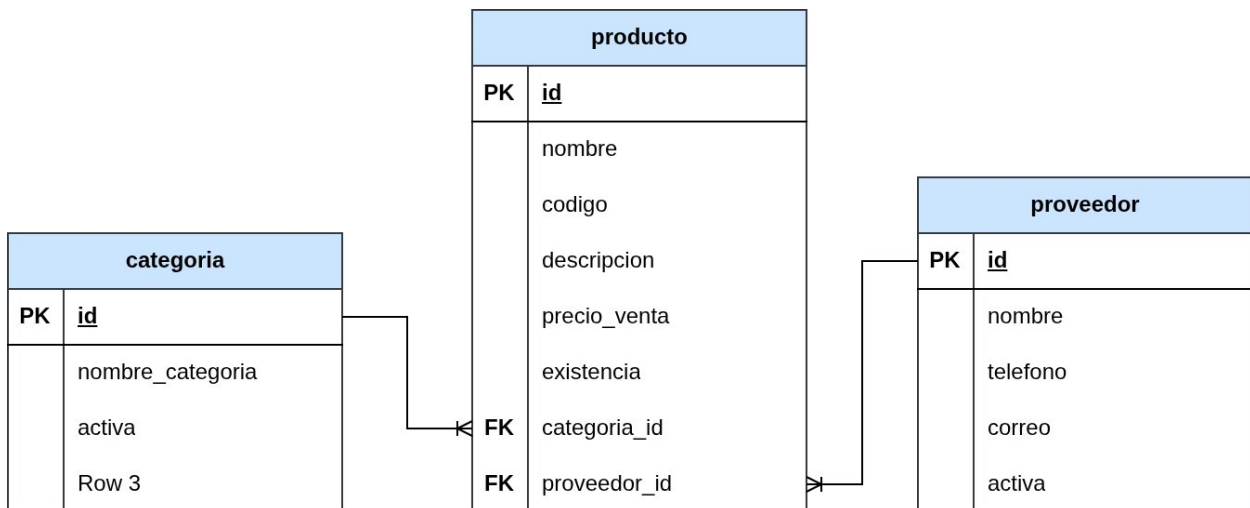
    @Column(name = "nombre") 2 usages
    private String nombre;

    @Column(name = "telefono") 2 usages
    private String telefono;

    @Column(name = "correo") 2 usages
    private String correo;

    @Column(name = "activo") 2 usages
    private Boolean activo;
```

Diagrama de relaciones



Scripts

V1__crear_tablas.sql

```
CREATE TABLE categoria (  
    id SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    activa BOOLEAN NOT NULL DEFAULT TRUE  
);
```

```
CREATE TABLE producto (  
    id SERIAL PRIMARY KEY,  
    codigo VARCHAR(30) NOT NULL UNIQUE,  
    nombre VARCHAR(150) NOT NULL,  
    precio_venta NUMERIC(12,2) NOT NULL,  
    existencia INTEGER NOT NULL DEFAULT 0,  
    categoria_id INTEGER NOT NULL,  
  
    CONSTRAINT fk_producto_categoria  
        FOREIGN KEY (categoria_id)  
        REFERENCES categoria(id)  
);
```

V2__crear_tablas.sql

```
ALTER TABLE categoria RENAME nombre to nombre_categoria;
```

V3__agregar_descripcion_producto.sql

```
ALTER TABLE producto ADD COLUMN descripcion VARCHAR(500);
```

V4__crear_tabla_proveedor.sql

```
CREATE TABLE proveedor (  
    id SERIAL PRIMARY KEY,  
    nombre VARCHAR(150) NOT NULL,  
    telefono VARCHAR(8) NOT NULL,  
    correo VARCHAR(50) NOT NULL,  
    existencia INTEGER NOT NULL DEFAULT 0,  
    activa BOOLEAN NOT NULL DEFAULT TRUE  
);
```

```
ALTER TABLE producto ADD COLUMN proveedor_id INTEGER;
```

```
ALTER TABLE producto
```

```
ADD CONSTRAINT fk_producto_proveedor
```

```
FOREIGN KEY (proveedor_id)
```

```
REFERENCES proveedor(id);
```

V5__renombrar_columna_activa_proveedor.sql

```
ALTER TABLE proveedor RENAME activa to activo;
```

Capturas Postman

GET http://localhost:8080/api/categorias

Params Body Headers Auth Vars Script Assert Tests Docs File Settings History

Query

Name	Value	Description
Name	Value	Description

Path ⓘ Bulk Edit

Response Headers³ Timeline Tests

{} JSON 200 OK 9ms 164B

```
1 {
2   {
3     "activa": true,
4     "id": 2,
5     "nombreCategoria": "Computadoras"
6   },
7   {
8     "activa": true,
9     "id": 3,
10    "nombreCategoria": "Accesorios"
11  },
12  {
13    "activa": true,
14    "id": 4,
15    "nombreCategoria": "Monitores"
16  }
17 }
```

GET http://localhost:8080/api/productos

Params Body Headers Auth Vars Script Assert Tests Docs File Settings History

Query

Name	Value	Description
Name	Value	Description

Path ⓘ Bulk Edit

Response Headers⁵ Timeline Tests

{} JSON 200 OK 12ms 653B

```
12 "nombre": "Laptop Lenovo",
13 "precioVenta": 850.00,
14 "proveedor": null
15 },
16 {
17   "categoria": {
18     "activa": true,
19     "id": 2,
20     "nombreCategoria": "Computadoras"
21   },
22   "codigo": "LAP-002",
23   "descripcion": null,
24   "existencia": 10,
25   "id": 4,
26   "nombre": "Laptop HP",
27   "precioVenta": 850.00,
28   "proveedor": null
29 },
30 {
31   "categoria": {
32     "activa": true,
33     "id": 4,
34     "nombreCategoria": "Monitores"
35   },
36   "codigo": "LCD-001",
37   "descripcion": null,
38   "existencia": 10,
39   "id": 5,
40   "nombre": "Monitor HP",
41   "precioVenta": 450.00,
42   "proveedor": {
43     "activo": true,
44     "correo": "hp@hp-mail.com",
45     "id": 2,
46     "nombre": "HP",
47     "telefono": "81000000"
48   }
49 }
50 }
```

GET http://localhost:8080/api/proveedores

Params Body Headers Auth Vars Script Assert Tests Docs File Settings History

Query

Name	Value	Description
Name	Value	Description

Path ⓘ Bulk Edit

Response Headers⁵ Timeline Tests

{} JSON 200 OK 15ms 86B

```
1 {
2   {
3     "activo": true,
4     "correo": "hp@hp-mail.com",
5     "id": 2,
6     "nombre": "HP",
7     "telefono": "81000000"
8   }
9 }
```

Evidencia PostgreSQL

```
gestion_productos=# select * from flyway_schema_history;
```

installed_rank	version	description	type	script	checksum	installed_by	installed_on	execution_time	success
1	1	crear tablas	SQL	V1__crear_tablas.sql	545171988	postgres	2026-09-28 19:42:38.879926	11	t
2	2	crear tablas	SQL	V2__crear_tablas.sql	-33426896	postgres	2026-09-28 19:58:24.143033	8	t
3	3	agregar descripcion producto	SQL	V3__agregar_descripcion_producto.sql	359864786	postgres	2026-09-28 20:12:51.511441	6	t
4	4	crear tabla proveedor	SQL	V4__crear_tabla_proveedor.sql	-165352175	postgres	2026-09-29 11:14:02.568401	17	t
5	5	renombrar columna activa proveedor	SQL	V5__renombrar_columna_activa_proveedor.sql	896307446	postgres	2026-09-29 11:16:22.779542	7	t

(5 rows)


```
gestion_productos=# select * from producto;
```

id	codigo	nombre	precio_venta	existencia	categoria_id	descripcion	proveedor_id
3	LAP-001	Laptop Lenovo	850.00	10	2		
4	LAP-002	Laptop HP	850.00	10	2		
5	LCD-001	Monitor HP	450.00	10	4		2

(3 rows)


```
gestion_productos=# select * from categoria
```

id	nombre_categoria	activa
2	Computadores	t
3	Accesorios	t
4	Monitores	t

(3 rows)


```
gestion_productos=# select * from proveedor;
```

id	nombre	telefono	correo	existencia	activo
2	HP	81000000	hp@hp-mail.com	0	t

(1 row)

Respuestas de comprobación

Comprobación de aprendizaje

1. ¿Cuál es la función de Spring Data JPA?

Spring Data JPA facilita la implementación de la Java Persistence API (JPA) como parte de un proyecto de Spring. Esto lo logra mediante la exposición de interfaces como `JpaRepository` que simplifica las consultas a tablas relacionadas a las entidades que Spring maneja, anotaciones como `@Transactional` que convierten los métodos de escritura en operaciones atómicas a las que se puede hacer rollback en caso de fallar y facilitando paginar los resultados de las consultas de lectura.

2. ¿Qué función cumple Hibernate?

Hibernate es el ORM, es decir la dependencia que se encarga de interpretar las clases de Spring marcadas con `@Entity` a registros en las tablas existentes en la base de datos y viceversa (las tablas son especificadas en cada clase mediante la anotación `@Table`). Lo anterior permite crear el esquema a partir de las definiciones en las clases de Spring, actualizar dicho esquema, gestionar su creación y eliminación, o bien, validar que las entidades de Spring corresponden a lo que ya existe en la base de datos.

3. ¿Qué diferencia existe entre JPA y Hibernate?

Funcionan en diferentes capas de la arquitectura, y, por ende, cumplen distintos propósitos, pues mientras JPA facilita la implementación de la lógica en la capa de acceso de datos, Hibernate se encarga de gestionar la traducción de entidades de Spring (clases de Java anotadas con `@Entity`) a registros de tablas en un motor de base de datos SQL.

4. ¿Qué función cumple `@Entity`?

`@Entity` indica a Spring que la clase anotada corresponde a una tabla en la base de datos utilizada por el proyecto. Esto permite que la clase sea utilizada por interfaces que extiendan de `JpaRepository` para la lectura y escritura en BD, así como que Spring gestione el ciclo de vida de estos objetos mediante JPA / Hibernate.

5. ¿Qué función cumple **@ManyToOne**?

Establece una relación muchos a uno en un miembro de una clase anotada con **@Entity**. De este modo, se indica que múltiples registros / instancias de la clase que posee la relación pueden asociarse a un único registro / instancia de la clase referenciada.

6. ¿Qué función cumple **@JoinColumn**?

Indica a Spring que la columna asociada al miembro anotado corresponde a una Foreign Key a nivel de la base de datos.

7. ¿Qué función cumple **JpaRepository**?

JpaRepository es una interfaz definida por Spring Data JPA que, aparte de definir métodos por defecto como `findAll()`, `findById()`, `save()` y `delete()`, utiliza un mecanismo llamado Query Derivation para permitir consultas específicas para la entidad que se asocia, permitiendo filtrar por campos, ordenar en base a los mismos, entre otras, a su vez que para consultas más complejas, permite usar JPQL (Java Persistence Query Language) mediante la anotación **@Query**.

8. ¿Por qué se utilizan migraciones?

Porque estas permiten llevar un control de los cambios que se han realizado sobre la base de datos, así como tener una única fuente de la verdad que especifique cómo se define el esquema actual. A su vez, facilita el deshacer o reaplicar cambios.

9. ¿Qué diferencia existe entre V1, V2 y V3?

V1 es el script inicial para la creación de las tablas con sus restricciones, V2 agrega la columna descripción a la tabla producto, y V3 crea la tabla proveedor, así como el constraint de clave foránea de la tabla producto a proveedor.

10. ¿Qué problema puede producir una relación bidireccional al generar JSON?

Esto crea una referencia circular, pues los objetos se anidan de manera recursiva e infinita, siendo que A referencia y contiene a B, B a su vez referencia y contiene a A, y así sucesivamente.